Atividade 06 — Delivery Tracker: Motoristas, Contratos de Repository e DI

Disciplina: Programação Web II — Backend (Node/Express) · IFAL/Maceió **Capítulo:** 4 — Repository Pattern e Injeção de Dependência **Modalidade:** Individual · **Peso:** parte do Exercício Cap. 4 (avaliado) **Correção:** automática (autograder de conformidade) + arquitetura

> **Pré-requisito:** Atividade 05 concluída (Entregas em camadas Controller → Service → Repository, persistência em memória).

Contexto

A operação de logística cresceu: agora cada entrega pode ser **atribuída a um motorista** cadastrado. O Tech Lead identificou um problema na Atividade 05 — pessoas do time acessavam os dados "por fora" dos repositories. A decisão: **todo repository expõe um contrato explícito**, e **nenhum service depende de implementação concreta** — só do contrato.

Objetivos de aprendizagem

- Aplicar o **Repository Pattern** como abstração de persistência.
- Programar contra **contratos (interfaces)** — inversão de dependência.
- Compor múltiplos repositories num **único ponto**.

Convenções

Valem as mesmas da Atividade 05 (base `/api`, erro `{erro}`, `GET /api/health`, `npm start` / `PORT` / `main`). Você continua **no mesmo repositório** `pweb2-delivery-<matricula>` criado a partir do modelo engsoft-ifal/pweb2-delivery-cap04 na Atividade 05 — apenas evolui o código. **Todas as rotas de Entregas continuam funcionando sem mudança de comportamento.**

Códigos de status usados aqui: `201` criado · `409` **unicidade** (CPF duplicado) · `422` **regra de estado/negócio** (atribuição inválida) · `404` não encontrado.

Modelo de domínio

Motorista

Campo	Tipo	Observação
`id`	number	gerado
`nome`	string	obrigatório
`cpf`	string	obrigatório; **único**
`placaVeiculo`	string	opcional
`status`	enum	`ATIVO` (padrão) | `INATIVO`

Na entrega, o campo `motoristaId` passa a ser preenchido quando há atribuição.

Requisitos funcionais

- **RF-01 — Cadastro de motorista:** `POST /api/motoristas` com `{ nome, cpf }` (e `placaVeiculo` opcional) → `201`, status `ATIVO`. CPF já cadastrado → `409` com mensagem descritiva.
- **RF-02 — Atribuir motorista a entrega:** `PATCH /api/entregas/:id/atribuir` com `{ motoristaId }`:
- só se a entrega estiver `CRIADA` (senão `422`);
- só se o motorista estiver `ATIVO` (senão `422`);
- registra evento no histórico da entrega.
- **RF-03 — Entregas de um motorista:** `GET /api/motoristas/:id/entregas` retorna **apenas** as entregas daquele motorista; o filtro `?status=` deve funcionar em combinação.
- **RF-04 — Contratos de Repository:** cada repository tem seu contrato documentado (JSDoc); os **services dependem só do contrato**, e a **composição ocorre num único ponto**.

Rotas obrigatórias (novas)

Método	Rota	Corpo	Sucesso	Erros
`POST`	`/api/motoristas`	`{ nome, cpf }`	`201` motorista (status `ATIVO`)	`400` campos · `409` CPF duplicado
`GET`	`/api/motoristas`	—	`200` array	—
`GET`	`/api/motoristas/:id`	—	`200` motorista	`404`
`GET`	`/api/motoristas/:id/entregas`	—	`200` só as entregas do motorista	`404`
`GET`	`/api/motoristas/:id/entregas?status=CRIADA`	—	`200` filtro combinado	`404`
`PATCH`	`/api/entregas/:id/atribuir`	`{ motoristaId }`	`200` entrega (com `motoristaId`)	`404` · `422` entrega não-CRIADA ou motorista INATIVO

Contratos de Repository (documente em JSDoc)

```
// IEntregasRepository
listarTodos(filtros?)  → Entrega[]
buscarPorId(id)        → Entrega | null
criar(dados)           → Entrega
atualizar(id, dados)   → Entrega
```

```
// IMotoristasRepository
listarTodos()          → Motorista[]
buscarPorId(id)        → Motorista | null
buscarPorCpf(cpf)      → Motorista | null
criar(dados)           → Motorista
```

Composição de dependências (ponto único)

Nunca `new` de repository dentro de service/controller. Só no *composition root*:

```
const database          = new Database();
const entregasRepo      = new EntregasRepository(database);
const motoristasRepo    = new MotoristasRepository(database);
const entregasService   = new EntregasService(entregasRepo, motoristasRepo);
const motoristasService = new MotoristasService(motoristasRepo, entregasRepo);
```

> A regra "motorista INATIVO não pode ser atribuído" mora no **EntregasService** (é regra da atribuição), não no MotoristasService.

Como a arquitetura é verificada

Além do autograder (comportamento), o critério de **inversão de dependência** é testado assim: **o professor substitui a implementação do repository por um `Mock` que respeita o mesmo contrato.** Se o sistema continuar funcionando → arquitetura correta. Se quebrar (`TypeError`/comportamento errado) → o service está acoplado à implementação concreta (critério de arquitetura zerado). Programe contra o contrato desde o início.

Correção automática (autograder)

Com a Atividade 06 concluída, o autograder deve atingir **100%** (Entregas + Motoristas):

```
npm start
BASE_URL=http://localhost:3000 node autograder/check.mjs   # meta: 122/122
```

A cada `push`, o **GitHub Actions** roda a correção e mostra a nota na aba **Actions**.

Entregável

- O mesmo repositório `pweb2-delivery-<matricula>` (branch `main`), agora com Motoristas + contratos + DI.
- `README.md` atualizado com as novas rotas e um diagrama simples (ASCII aceito) da composição das dependências.

Critérios (transparentes)

Critério	Peso
Autograder — conformidade (Entregas + Motoristas)	50%
Services dependem só dos contratos (passa no *swap* por Mock)	25%
Contratos documentados (JSDoc) cobrindo os métodos obrigatórios	10%
Composição num único ponto (sem `new` em service/controller)	10%
README com rotas e diagrama da composição	5%